

AI 활용 기술 문서

AI에게 코드를 시키는 대신, AI를 채점할 기계를 먼저 만들었습니다.

프로젝트 PULSEFALL

팀 김범수 · 송채우 · 신승민

제출일 2026. 08. 10

이 문서의 요지

AI에게 "게임 만들어줘"라고 하면 게임은 나옵니다. 문제는 그게 재미있는지, 공정한지, 60fps로 도는지를 AI 자신도 모른다는 것입니다. 저희가 실제로 설계한 것은 게임 코드가 아니라 **AI의 출력을 사람 손을 거치지 않고 기각할 수 있는 검증 장치**였고, 게임은 그 장치를 통과한 결과물입니다.

01 사용한 AI 도구

도구	사용 범위	비중
Claude Code (Anthropic, Claude Opus)	게임 전체 구현, 리팩터링, 차트 공정성 솔버 설계, 오토플레이 봇 작성, 성능 프로파일링, 문서 작성. 터미널에서 파일을 직접 읽고 쓰며, node 스크립트를 실행해 결과를 보고 스스로 수정하는 방식.	코드 작성의 대부분
헤드리스 브라우저 자동화 (Chrome DevTools Protocol)	실제 렌더 프레임 계측, 카메라 각도 측정, 뷰포트별 레이아웃 검증. AI가 "구현했다"고 주장한 값을 실행 중인 화면에서 다시 재서 확인 .	검증 전용

생성형 이미지·음악 AI는 일절 사용하지 않았습니다. 그래픽은 전부 Canvas 2D 코드이고, 음악은 Web Audio API로 브라우저에서 실시간 합성합니다 (§6 참조).

02 사람이 한 일 / AI가 한 일

사람이 결정한 것

- 무엇이 재미인가 — "피하는 게임"을 "붙어 있는 게임"으로 뒤집는 결정
- 무엇을 금지할 것인가 — 성능·공정성에 대한 불변 규칙
- 무엇을 합격으로 볼 것인가 — 자동 검사의 통과 기준
- 어디까지 허용할 것인가 — 카메라 45° 상한 같은 수치 한계
- AI의 보고를 믿지 않고 다시 재는 일

AI가 수행한 것

- 약 **6,700줄**의 구현 (게임 5,400줄 + 도구·서버 1,300줄)
- 차트 공정성 솔버와 오토플레이 봇의 알고리즘 구현
- 실패한 검사를 읽고 **원인을 역추적해 스스로 수정**
- 성능 계측 스크립트 작성 및 실행
- 문서화

03 AI에게 준 상시 지시 — "고정된 헌법"

매번 프롬프트로 반복하는 대신, **여기면 자동으로 빌드가 깨지는 형태로** 규칙을 코드에 심었습니다. AI는 이 규칙들을 지키는 코드만 통과시킬 수 있습니다.

지시	강제 수단
"ctx.shadowBlur를 절대 쓰지 마라"	그림자 블러 한 번이 캔버스 전면 블러 패스를 유발해 모바일에서 프레임을 무너뜨립니다. 검사 스크립트가 소스 전체를 훑어 사용 시 즉시 실패 시킵니다. (주석은 제외하고 코드만 검사 — 금지 이유를 설명한 주석까지 걸리면 안 되므로)
"봇이 못 깨는 차트는 차트가 아니다"	헤드리스 오토플레이 봇이 모든 차트를 100% 클리어 하지 못하면 배포가 막힙니다. GitHub Actions에서 push마다 실행됩니다.
"연출이 판정을 건드리면 안 된다"	카메라·기울기·흔들림은 전부 읽기 전용 으로 시뮬레이션을 참조만 합니다. 시뮬레이션은 240Hz 고정 tick으로 렌더와 완전히 분리돼 있습니다.
"박자 위의 PULSE는 절대 거부하지 마라"	게임 로직·차트 검증기·봇 세 곳이 같은 규칙을 공유 해야 합니다. 한 곳이라도 어긋나면 §5의 사고가 재발합니다.

04 구조 — 검증 가능하게 만든 설계

4.1 결정론이 검증의 전제

시뮬레이션을 **240Hz 고정 tick**으로 돌리고 렌더링과 분리했습니다. 같은 입력은 항상 같은 결과를 내므로, **봇이 클리어한 차트는 사람도 반드시 클리어**할 수 있습니다. 이 성질이 없으면 아래 모든 자동 검증이 무의미해집니다.

4.2 차트 공정성 솔버 — "불가능한 배치를 컴파일 타임에 제거"

절차 생성된 벽 배치는 그냥 두면 **물리적으로 통과 불가능한 조합**이 나옵니다. 차트 컴파일러는 벽을 도착 시각 기준으로 **링(ring)**으로 묶고, 플레이어의 **실제 각속도로** 다음 틸까지 갈 수 있는지 계산합니다.

```
// 속도가 섞인 패턴에서는 나중에 생성된 링이 먼저 도착할 수 있다.
// 플레이어는 도착 순서로 만나므로, 솔버도 그 순서로 걸어야 한다.
rings.sort((a, b) => a.enter - b.enter);
```

달을 수 없는 링은 **격자 위에서 회전시켜** 반드시 도달 가능하게 만듭니다. 틸이 아예 없는 SEAL 링은 회전으로 풀 수 없으므로, **PULSE 쿨다운을 모델링**해 "직전 PULSE 이후 충분한 시간이 지났는가"까지 확인합니다. 실측 예 — HOLLOW SUN: 링 173개 중 37개를 회전 보정, 최소 여유 0.0437.

4.3 오토플레이 봇 — CI에서 매번 전곡을 플레이

봇은 매 tick마다 눈앞의 벽을 180개 각도 샘플로 스캔해 빈 구간을 찾고, 남은 시간 안에 **도달 가능한지** 판단해 이동하거나 PULSE를 씹니다. push할 때마다 3개 고정 곡 + DAILY + ENDLESS 6시드를 전부 플레이합니다.

79

자동 검사

100%

전 차트 봇 클리어

6/6

ENDLESS 시드 생존

530

검증된 벽 (고정 곡)

05 대표 사례 — AI가 만든 검사기가 거짓말을 한 날

이 프로젝트에서 **AI 디렉팅이 실제로 값을 한 순간**이라 따로 적습니다.

증상

오토플레이 봇이 HOLLOW SUN의 **38.0%** 지점에서 죽고, "이 차트는 클리어 불가능"이라고 보고했습니다. 가장 자연스러운 다음 행동은 **차트 생성기를 고치는** 것이었습니다.

사람이 개입한 지점

차트를 고치기 전에 "**솔버는 뭐라고 하는가**"를 먼저 물었습니다. 진단 스크립트를 따로 짜서 돌린 결과, 공정성 솔버는 **모든 링이 도달 가능하다고** 답했습니다 (`worstSlack 0.0437 > 0`). 즉 **두 도구의 주장이 서로 모순**이었습니다.

원인

사망 프레임을 덤프하는 스크립트를 짜서 죽는 순간을 그대로 찍어봤습니다.

```
t=41.83 charge=0.18 group=1 free=0/180 eta=0.21 reachable=false
walls: side=0 span=8/8 kind=seal ← 틈이 없는 SEAL 링
```

봇은 SEAL 앞에서 **PULSE를 요청조차 하지 않고** 가만히 서서 죽고 있었습니다. 봇의 게이트가 `charge >= 0.32`를 요구했기 때문입니다. 그런데 게임의 실제 규칙은 다릅니다.

```
// World._updatePulse - 박자 위에서는 게이지와 무관하게 통과시킨다.
if (perfect || this.charge >= cost - 1e-6) { ... }
```

게임은 허락하는데 봇이 스스로 금지하고 있었습니다. 게이지 0.18로도 비트만 맞으면 뚫을 수 있는 벽 앞에서, 봇은 자기가 만든 더 엄격한 규칙 때문에 포기한 것입니다.

이 실패가 위협했던 이유

봇의 보고는 "**차트가 불공정하다**"였습니다. 그 말을 믿고 차트 생성기를 고쳤다면, **멀쩡한 차트를 망가뜨리면서 진짜 버그는 그대로 남았을** 것입니다. 자동화된 검사가 틀렸을 때 가장 비싼 대가는 검사가 실패하는 것이 아니라 **엉뚱한 파일을 고치게** 만드는 것입니다.

조치

봇의 게이트를 게임의 게이트와 일치시키고, 다시는 갈라지지 않도록 **주석으로 근거를 못박**았습니다.

```
// 여기의 게이트는 월드의 게이트여야 하며, 그것을 더 엄격하게 추측한 값이면 안 된다.
// 월드는 박자 위의 PULSE를 절대 거부하지 않는다 (ONBEAT_ALWAYS_ALLOWED).
// 그 규칙이 SEAL을 공정하게 만드는 근거 전부다.
const onBeat = world.beatError() <= BEAT_WINDOW;
if (group > 0 && (bestIdx < 0 || !reachable) && world.pulseTimer <= 0) {
  if ((onBeat || world.charge >= PULSE_COST) && eta < 0.34) world.requestPulse();
}
```

결과: 실패하던 2개 차트가 **전부 100%**가 되었습니다. 차트 코드는 한 줄도 고치지 않았습니다.

여기서 얻은 원칙

AI가 만든 검증 도구도 **검증 대상**입니다. 서로 다른 두 도구가 모순된 주장을 하면, 둘 중 하나가 틀렸다는 사실 자체가 가장 **중요한 신호**입니다. 그래서 저희는 실패 보고를 받았을 때 "무엇을 고칠까"보다 "누가 거짓말하고 있나"를 먼저 묻는 절차를 두었습니다.

06 주요 프롬프트 및 지시 사항

실제로 입력한 지시를 그대로 옮깁니다. 요약·윤색하지 않았습니다.

㉠ 방향 전환 — 재미없다는 진단에서 시작

"너~~무 재미없고 렉걸리고, 지루하고, 뻘하고 루즈하고 노잼"

기존 프로토타입에 대한 최초 피드백

지시의 성격: 해결책을 지정하지 않고 **증상만** 전달했습니다. "렉"과 "지루함"은 서로 다른 층위의 문제이므로, AI에게 먼저 **원인을 분리해 진단하도록** 시켰습니다. 그 결과 두 갈래로 나뉘었습니다 — 성능은 shadowBlur 남용, 재미는 **"피하는 게임에는 잘할 이유가 없다"**는 구조적 문제. 후자에서 **GRAZE = 스칠수록 강해진다**는 지금의 중심 규칙이 나왔습니다.

㉡ 연출 지시 — 수치 상한을 함께 준다

"여기서 게임 화면 각도를 2D가 아니라, **최대 45도까지** 기울어지도록 (시간 지났을때, 어려운 구간에서) 구현해줘. 더 모션 웅장 번쩍 재밌게 수정"

지시의 성격: "멋있게"라는 모호한 요구에 **검증 가능한 상한(45°)**과 **변화의 조건(시간·난이도)**을 함께 넣었습니다. 덕분에 "구현했다"는 주장을 **실제 화면에서 각도를 재서** 확인할 수 있었습니다.

구간	속도	실측 기울기
RUNWAY (도입)	0.88	7.7°
VECTOR	1.30	16.5°
PRESSURE	1.50	25.3°
DROP	1.72	36.3°
TERMINAL	2.05	45.0°

초기 구현은 연출 충격(kick)이 겹칠 때 **약 53°까지 넘어갔습니다**. 지시는 "최대 45도"였으므로 **하드 상한으로 다시 조였고**, 모든 충격을 동시에 발생시켜도 45.00°에서 멈추는 것을 확인했습니다.

㉢ 성능 지시 — 금지 목록을 먼저 준다

"ctx.shadowBlur는 프로젝트 전체에서 금지. 검사 스크립트가 잡아내게 만들어라."

지시의 성격: "최적화해줘"가 아니라 **단일 금지 규칙 + 자동 강제**. AI에게 판단을 맡기면 매번 다르게 판단하지만, 검사 스크립트는 매번 같게 판단합니다.

㉣ 공정성 지시 — 통과 기준을 숫자로

"오토플레이 봇을 만들어서 모든 차트를 100% 클리어하는지 CI에서 확인해라. 하나라도 못 깨면 실패로 처리."

지시의 성격: 재미는 자동 판정이 불가능하지만 **공정성은 가능합니다**. 판정 가능한 것만 자동화하고, 나머지에 사람의 시간을 남겼습니다.

㉔ **진단 지시 — 고치기 전에 원인을 요구**

"봇이 왜 죽는지 **먼저 증명해라**. 추측으로 차트 건드리지 말고, 죽는 프레임을 그대로 찍어서 보여줘."

지시의 성격: §5의 사고를 막은 지시입니다. AI는 실패를 보면 즉시 수정하려는 경향이 있어서, "**수정 금지, 증거 먼저**"를 명시적으로 요구해야 했습니다.

㉕ **복구 지시 — 세션 기록에서 삭제된 빌드를 되살리기**

"이때 말했던 버전 게임 다시 플레이할 수 있게 보내줘"

지시의 성격: 이 버전은 이후 실험 과정에서 **덮어써져 디스크에 존재하지 않았고, git 이력도 없었습니다**. AI에게 **자기 자신의 작업 기록(세션 트랜스크립트)에 남은 파일 쓰기·편집 호출을 순서대로 재생**하도록 지시해 삭제된 빌드를 복원했습니다. 100개 작업 중 98개가 자동 적용되었고, 나머지는 손으로 메웠습니다. 이 경험이 **지금 이 저장소에 git 이력을 남기는 이유**입니다.

07 외부 에셋 · 오픈소스 출처

결론
외부 에셋 파일과 서드파티 라이브러리를 하나도 사용하지 않았습니다. package.json에 dependencies 항목이 존재하지 않으며, node_modules 없이 실행됩니다.

항목	출처 / 라이선스
런타임 라이브러리	없음. 브라우저 표준 API만 사용 — ES Modules, Canvas 2D, Web Audio API, Gamepad API, MediaRecorder API.
빌드 도구	없음. 번들러·트랜스파일러·전처리기를 쓰지 않습니다.
이미지 · 스프라이트	없음. 모든 그래픽은 런타임에 Canvas 2D 코드로 그립니다. 앱 아이콘도 저장소의 scripts/make_icons.mjs가 생성합니다.
사운드 · 음악	없음. 오디오 파일이 0개입니다. 드럼·베이스·아르페지오·패드·리버브·딜레이를 Web Audio 노드로 실시간 합성 합니다. (src/audio/)
폰트	웹폰트 미탑재. OS 기본 시스템 폰트와 모노스페이스 폰트만 지정합니다.
CI 액션	GitHub 공식 액션 — actions/checkout, setup-node, configure-pages, upload-pages-artifact, deploy-pages (MIT). 배포 파이프라인 전용이며 게임 실행물에 포함되지 않습니다.
게임 코드	전량 본 팀 자체 작성 (AI 도구 활용). MIT 라이선스로 저장소에 공개.

창작 범위

게임 규칙, 차트, 그래픽, 음악, 코드 전부 본 팀이 자체 설계·작성했습니다. 타사 게임의 코드·에셋·차트 데이터를 참조하거나 사용한 부분이 없습니다. 중심 규칙인 **GRAZE 자원 경제**(스칠수록 강해지고 안전 플레이는 0점)와 **박자 위에서만 열리는 SEAL**은 본 팀의 고유 설계입니다.

08 정리 — 저희가 주장하는 것

- **AI에게 맡긴 것은 구현이고, 사람이 줬 것은 판정 기준입니다.** "무엇을 합격으로 볼 것인가"를 먼저 정의하고, 그 정의를 코드로 강제했습니다.
- **자동 판정이 가능한 것만 자동화했습니다.** 공정성·성능·API 금지는 기계가 보고, 재미는 사람이 봅니다. 이 경계를 흐리지 않은 것이 속도의 이유입니다.
- **AI의 자기 보고를 신뢰하지 않았습니다.** "구현했다"는 문장 대신 실행 중인 화면에서 썬 숫자를 받았고, 그 습관이 §5의 오진을 막았습니다.
- **AI가 만든 도구도 의심 대상에 포함시켰습니다.** 검사기가 실패를 보고했을 때 제품이 아니라 **검사기**가 틀렸던 사례가 실제로 있었고, 그것을 잡아낸 것이 이 프로젝트에서 가장 값이 나갔던 판단입니다.